

MiniLang Compiler Design Report

Course: Compiler Design Lab | Student Roll: 241030042 | Variant: 2 (Structs & Nested Functions with Static Scoping)

1. Design Decisions

1.1 Hand-written DFA Lexer

Instead of generator tools like Lex/Flex, the lexer was implemented by hand using a Deterministic Finite Automaton (DFA) in `src/lexer.c`. It maintains line and column tracking counters to report exact coordinates for lexical errors. Tokens and keywords follow the roll-based transformation rule: reversing the standard English keyword and suffixing with 42 (e.g., `tcurts42` for `struct`, `noitcnuf42` for `function`).

1.2 Recursive-Descent LL(1) Parser & AST

The parser is built without generator tools using top-down recursive descent. Left recursion in expression productions (e.g., $E \rightarrow E + T$) was eliminated by rewriting rules into tail-recursive form ($E \rightarrow T E'$, $E' \rightarrow + T E' \mid \epsilon$). Left-factoring was systematically applied to eliminate ambiguity in conditionals (dangling else) and postfix expressions (field access vs. calls). The parser builds an Abstract Syntax Tree (AST) representing declarations, statements, struct types, and nested functions.

1.3 Static Scoping & Scoped Symbol Table

Variant 2 requires nested functions with static (lexical) scoping. The symbol table is implemented as a tree of Scope nodes, each having a parent link pointing to its enclosing lexical scope. Variable and function lookup walks up the parent link chain. The semantic analyzer enforces static scoping and checks 5 distinct semantic errors: (1) Undeclared identifiers, (2) Duplicate declarations in the same scope, (3) Assignment/arithmetic type mismatches, (4) Invalid struct field access, and (5) Return type mismatches.

1.4 Three-Address Code (TAC) & Optimizations

The AST is lowered into linear Three-Address Code (TAC) instructions. Two optimizations are implemented: **(a) Constant Folding:** Evaluates constant arithmetic and comparison expressions at compile-time (e.g., $t1 = 10 + 20 \rightarrow t1 = 30$). **(b) Dead Code Elimination:** Removes unreachable instructions following unconditional jumps (`goto`, `return`) and unused temporaries.

1.5 Backend: Stack-Machine & Virtual Machine Runner

The backend compiles TAC into stack-machine bytecode instructions (`PUSH_INT`, `STORE_VAR`, `LOAD_VAR`, `ADD`, `SUB`, `CALL`, `RET`, `ALLOC_STRUCT`, `GET_FIELD`, `SET_FIELD`). A virtual machine runner executes the bytecode with an activation stack and heap-allocated struct records.

2. A Bug Encountered & Debugging Process

The Bug: During testing of nested functions (`test06_nested_functions.ml`), the compiler reported that outer variables were 'undeclared' inside nested functions, and at runtime, parameters evaluated to 0 when functions were called.

Root Cause: (1) The parser initially stored nested functions in a separate list from statements, causing semantic analysis to check nested functions before outer local variables were registered in the scope. (2) The stack machine did not pop passed arguments into the callee's local frame upon function entry, violating calling convention.

Solution: (1) Unified the function body into a sequential block where nested functions are parsed in textual statement order, allowing them to resolve outer variables via the parent scope link. (2) Introduced a `TAC_PARAM_POP` opcode to pop arguments off the stack into parameter variables in reverse order (standard cdecl calling convention). (3) Emitted unconditional jumps (`goto L_skip`) around nested function bodies to avoid fall-through during outer function execution.

3. One Thing I Would Change

If I were to redesign or extend this compiler, I would **implement a Register Allocator (Linear Scan or Graph Coloring) to emit native x86-64 assembly instead of interpreted bytecode**. This would produce standalone native executables that run directly on the host CPU without needing a VM runtime. Furthermore, adding an automatic garbage collector (such as a mark-and-sweep collector) for heap-allocated struct objects would ensure memory safety for large-scale applications.